UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Simularea functionarii unei memorii cache

Structura Sistemelor de Calcul

Autor:Berciu Oana-Georgiana

Grupa:30234

Profesor indrumator:Bozdog Raluca

Cuprins:

1. Descriere proiect

2. Motivația alegerii proiectului

3. Planificarea proiectului

4. Studiu bibliografic

4.1. Tehnici de mapare cache

4.2. Politici de scriere

4.3. Politici de înlocuire

4.3. Măsurarea performanței

5. Analiza sistemului

5.1. Cazuri de utilizare

5.2. Funcționalități ale aplicației

5.2.1. Funcționalități principale

5.2.2. Setări de simulare

5.2.3. Validări și mesaje

5.3. Interacțiunea cu utilizatorul

6. Designul aplicației

6.1. Arhitectura și componentele sistemului

Nivel 1 – Designul general al sistemului

Nivel 2 – Divizarea în subsisteme (Model, Simulation, GUI)

Nivel 3 – Divizarea în clase și relațiile dintre ele

6.2. Interfața grafică

7. Implementarea aplicației

7.1. Librării utilizate

7.2. Clasele aplicației

7.2.1. Clasa CacheBlock

7.2.2. Clasa Cache

7.2.3. Clasa FifoCache

7.2.4. Clasa MainMemory

7.2.5. Clasa CacheSimulator

7.2.6. Clasa SimulationFrame

7.3. Arhitectura Aplicatiei

7.4. Algoritmi utilizati

7.4.1.Politica de inlocuire FIFO

7.4.2.Politica de inlocuire LRU

7.4.3. Politica de scriere Write-Through

7.4.4. Structura de tip Fully-Associative

7.5. Operatii fundamentale asupra memorie cache

7.5.1.Regasirea

7.5.2.Plasarea

7.5.3.Descarcarea

8. Testare

8.1 Metodologia de testare

8.2 Categorii de teste

8.2.1 Teste de citire

8.2.1.1 Rezultate asteptate

8.2.1.2 Rezultate obtinute pentru politica Fifo

8.2.1.3 Rezultate obtinute pentru politica Lru

8.2.1.4 Interpretarea rezultatelor

8.2.2 Teste de scriere

8.2.2.1 Rezultate asteptate

8.2.2.2 Rezultate obtinute pentru politica Fifo

8.2.2.3 Rezultate obtinute pentru politica Lru

8.2.3 Teste de invalidare

8.2.3.1 Rezultate asteptate

8.2.3.2 Rezultate obtinute pentru politica Fifo

8.2.3.3 Rezultate obtinute pentru politica Lru

9.Conluzie

10.Bibliografie

1.Descriere proiect

Simularea funcționării unei memorii cache Java implică crearea unei structuri de date care mimează comportamentul memoriei cache, implementând operații precum plasarea, regasirea și descarcarea datelor, gestionând simultan dimensiunea limită și eventual politici de evacuare.

2.De ce am ales acest proiect

Am ales sa implementez acest proiect deoarece combina concepte teoretice de arhitectura calculatoarelor cu partea practica de programare.Tema mi s-a parut interesanta pentru ca permite intelegerea modului in care procesorul gestioneaza datele si optimizeaza performanta.

De asemenea,proiectul include o componenta grafica ce ofera o reprezentare vizuala a principalelor operatii ale unei memorii cache-plasarea,regasirea si descarcarea informatiilor-facilitand intelegerea modului in care aceste procese se desfasoara in timp real.

Implementarea in Java este un avantaj,deoarece limbajul este portabil si clar,ceea ce face simularea mai intuitiva.

3.Planificare proiect

Săptămâna 3 (15 oct): propunerea proiectului, planificarea etapelor de lucru, realizarea studiului bibliografic privind memoriile cache și tehnicile de mapare

Săptămâna 5 (29 oct): implementarea claselor în Java pentru reprezentarea memoriei cache; definirea logicii de simulare (plasare, regăsire, descarcare), calculul ratei de hit/miss

Săptămâna 7 (12 nov): implementarea interfeței grafice (UI) și integrarea acesteia cu logica de simulare

Săptămâna 9 (26 nov): extinderea funcționalităților aplicației (selectarea politicilor de înlocuire, selectarea tipului de mapare – directă, set-asociativă etc.) și optimizări

Săptămâna 11 (10 dec): testarea și evaluarea aplicației pe mai multe configurații, înregistrarea rezultatelor, definitivarea documentației și pregătirea prezentării proiectului

4.Studiu bibliografic

Memoria cache reprezintă o componentă esențială a arhitecturii calculatoarelor moderne, având rolul de a reduce timpul de acces al procesorului la datele stocate în memoria principală (RAM). Aceasta funcționează pe baza principiilor de localitate temporală și spațială, adică presupune că datele accesate recent sau cele aflate în apropierea lor vor fi utilizate din nou în curând.

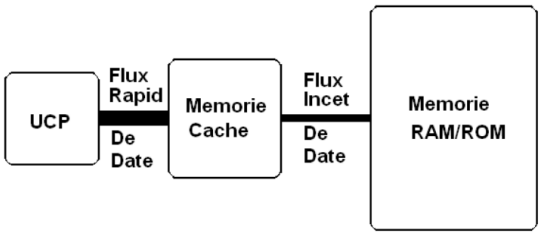


Figura 1: Sursa Wikipedia

Cache-ul este o tehnică de stocare temporară a unei copii a datelor într-o memorie rapid accesibilă. Cache-ul stochează cele mai recent utilizate cuvinte într-o memorie mică pentru a crește viteza de acces la date. Acesta acționează ca un tampon între RAM și CPU și astfel crește viteza cu care datele sunt disponibile procesorului.

O caracteristică esențială a memoriei cache este funcția de mapare, care stabilește corespondența dintre blocurile din memoria principală și locațiile din cache. Aceasta se poate realiza prin trei tehnici principale: mapare directă, mapare asociativă și mapare asociativă pe seturi.

4.1 Tehnici de mapare cache

1. Direct Mapped Cache

Maparea directă este o tehnică simplă și des utilizată pentru maparea memoriei cache, în care fiecare bloc din memoria principală este asociat cu exact o locație din cache, numită linie de cache (cache line). Dacă două blocuri de memorie se mapează pe aceeași linie de cache, unul îl va înlocui pe celălalt, ceea ce poate duce la cache misses. Performanța mapării directe este proporțională cu rata de hit .

2. Fully Associative Cache

În maparea complet asociativă, orice bloc din memoria principală poate fi stocat în orice linie din cache. Spre deosebire de maparea directă, unde fiecare bloc are o singură poziție posibilă, aici există o flexibilitate totală. Aceasta îmbunătățește rata de hit, dar necesită un mecanism mai complex de căutare și gestionare a liniilor din cache.

3. Set Associative Cache

Maparea set-asociativă este un compromis între maparea directă și cea complet asociativă. Această tehnică combină eficiența mapării directe cu flexibilitatea mapării asociative.

În acest tip de mapare, liniile de cache sunt grupate în seturi (de obicei 2, 4 sau mai multe linii per set).

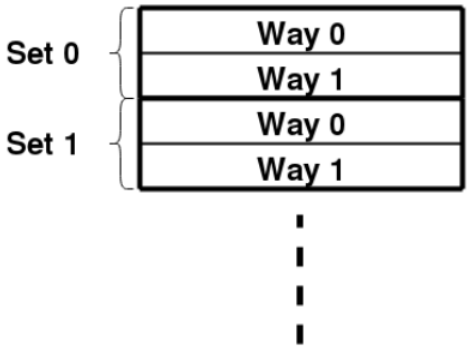


Figura 2: Sursa GeekForGeeks

4.2 Politici de scriere

Pentru menținerea coerenței datelor între cache și memoria principală sunt utilizate două politici majore:

- **Write-Through (Scriere directă)**
  - Datele sunt scrise simultan atât în cache, cât și în memoria principală.
  - Avantaj: memoria principală este întotdeauna sincronizată cu cache-ul, ceea ce reduce riscul de pierdere a datelor.
  - Dezavantaj: poate încetini performanța, deoarece fiecare scriere implică și accesul la memoria principală.
- **Write-Back (Scriere întârziată)**
  - Datele sunt scrise inițial doar în cache și sunt actualizate în memoria principală doar atunci când linia de cache este înlocuită.
  - Avantaj: reduce numărul de accesări ale memoriei principale, îmbunătățind performanța.
  - Dezavantaj: necesită un mecanism suplimentar pentru a urmări ce linii din cache au fost modificate (bitul “dirty”).

4.3 Politici de înlocuire

Când toate liniile/cache-seturile sunt ocupate, se decide eliminarea unui bloc pe baza unei politici de replacement. Cele mai comune sunt:

- **LRU (Least Recently Used)** – elimină datele cel mai puțin recent utilizate

- **FIFO (First In, First Out)** – elimină primul bloc introdus
- **Random Replacement** – alegere aleatorie (simplă, dar mai puțin eficientă)
- **LFU (Least Frequently Used)** – urmărește frecvența accesărilor

Alegerea politicii influențează puternic rata de hit a cache-ului.

4.4 Măsurarea performanței

Performanța cache-ului este evaluată prin indicatori precum:

- **Hit rate** – procentul de accesări care se regăsesc în cache
- **Miss rate** – accesări care necesită apelarea memoriei principale

5.Analiza

5.1.Cazuri de utilizare

Aplicația Simularea unei memorii cache este destinată utilizatorilor care doresc să observe modul de funcționare al unei memorii cache și impactul diferitelor configurări asupra accesărilor la memorie.

Interacțiunea utilizatorului cu aplicația se realizează printr-un set de acțiuni specifice, denumite cazuri de utilizare.

Actor principal: Utilizator

Acesta folosește aplicația pentru a configura simularea, a introduce datele necesare și a vizualiza rezultatele obținute.

Lista cazurilor de utilizare:

1. Configurare simulare- Utilizatorul pornește aplicația și poate configura parametrii memoriei cache: dimensiune(numar de linii), politica de înlocuire.

2. Introducere adrese- Utilizatorul poate introduce manual adresele ce vor fi accesate în timpul simulării sau poate utiliza un fisier text pentru a nu introduce fiecare adresa manual.

3. Rulare simulare- Utilizatorul pornește simularea, iar aplicația procesează accesările și determina numarul de HIT/MISS si calculeaza si ratele.

4. Vizualizare rezultate- Utilizatorul poate vizualiza starea cache-ului și statisticile: număr de hit-uri/miss-uri și procentajele aferente.

5. Resetare simulare- Utilizatorul poate reseta parametrii și datele pentru a realiza o nouă simulare.

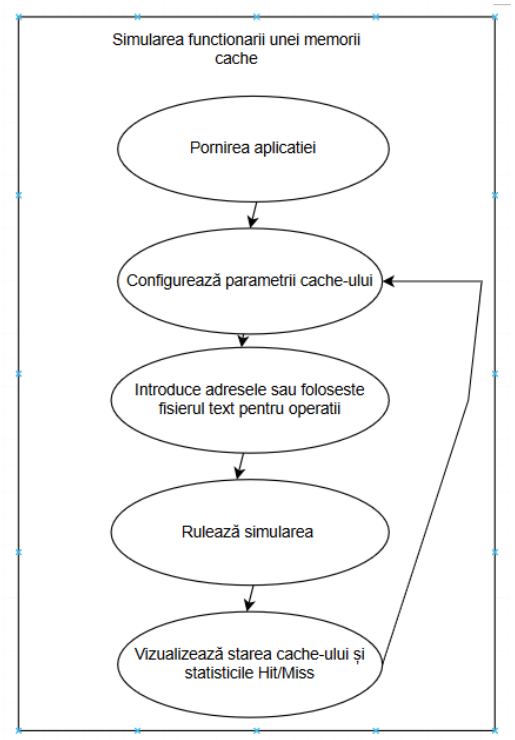


Figura 3.Diagrama de activitate

Figura 4:Diagrama Use Case

5.2 Funcționalități

Aplicația permite simularea simplificată a funcționării unei memorii cache, cu interfață grafică pentru introducerea datelor și vizualizarea rezultatelor.

5.2.1 Funcționalități principale

- **Pornire simulare**: rulează accesările la memorie introduse de utilizator.
- **Introducere date**: utilizatorul poate introduce manual, într-un câmp text (TextField), o adresa sau o valoarea pentru accesa sau a scrie date in memorie
- **Vizualizare cache în timp real**: afișare grafică sub formă de pătrate colorate:
    - verde = **HIT**, roșu = **MISS**; galben = **Scriere in memorii**
- **Vizualizare statistici intr-o pagina separata**:
    - **Hit Count**, **Miss Count**, **Hit Rate (%)**, **Miss Rate (%)**.
- **Reset**: șterge starea curentă a cache-ului si statisticile finale,revinind la ecranul de configurare.
- **Rulare din fisier**
- **Salvare rezultate in fisiere separate pentru cele 2 politici de inlocuire implementate:** intr-un fisier de rezultate se poate vedea tot ce s-a intamplat la fiecare pas si cum arata starea Cache-ului si a memoriei principale dupa fiecare operatie efectuata

5.2.2 Setări de simulare

- **Tip de mapare**: Fully Associative .
- **Politică de înlocuire**: FIFO,LRU .
- **Politică de scriere**: Write-Through .

Aceste configurații sunt setate implicit cand deschizi aplicatia,iar tipul politici de inlocuire poate fi modificat dintr-un comboBox,iar dupa ce este selectat trebuie resetata aplicatia,apasand pe butonul Reset.

5.2.3 Validări și mesaje

- Verifică formatul adreselor introduse; semnalează intrările invalide.
- Blochează butonul de rulare dacă lipsesc adrese sau parametri esențiali.
- Mesaje informative la finalul simulării (ex.: „Simulare finalizată”, „Nu există adrese valide” etc.)

5.3 Interactiunea cu utilizatorul

Interacțiunea utilizatorului cu aplicația reprezintă un element esențial în modul de utilizare și în înțelegerea funcționării memoriei cache. O interfață intuitivă contribuie direct la o experiență eficientă și la o utilizare corectă a aplicației, fără a necesita instrucțiuni suplimentare.

Utilizatorul interacționează direct cu fereastra principală a aplicației, în care se regăsesc elementele necesare pentru configurarea și rularea simulării. Pe ecranul principal sunt afișate:

- o zonă grafică destinată reprezentării vizuale a memoriei cache,
- o zonă grafică destinată reprezentării vizuale a memoriei principale in partea de jos a ferestrei si
- un buton pentru a accesa panoul de control

In panoul de control sunt afisate urmatoarele:

- un JComboBox pentru selectarea tipului de politica de inlocuire pe care vrei sa o folosesti,
- un câmp pentru introducerea numarului de linii ce vor fi folosite,
- un câmp pentru introducerea adreselor ce vor fi accesate,
- un camp pentru introducerea valorilor ce vor fi scrise,
- butoane pentru accesare,scriere,
- un buton pentru resetare,
- un buton ce deschide o secțiune pentru afișarea statisticilor si
- un buton pentru rularea din fisier a operatiilor ce vrem sa le efectuam

Etichetele butoanelor sunt clare și indică în mod direct acțiunea pe care utilizatorul o poate realiza, permițând astfel o navigare ușoară în cadrul aplicației.

În momentul în care utilizatorul pornește simularea, interfața își actualizează automat elementele vizuale: liniile cache-ului sunt evidențiate prin culori distincte (verde pentru HIT, roșu pentru MISS), iar statisticile sunt afișate într-un panou separat, pentru a facilita interpretarea rezultatelor.

După finalizarea unei simulări, utilizatorul poate modifica datele introduse sau poate efectua o resetare a aplicației pentru a rula un nou set de accesări. Astfel, utilizatorul poate relua procesul oricând, fără a fi necesară repornirea aplicației.

6.Design

6.1 Arhitectură și componente

Arhitectura aplicației poate fi împărțită în mai multe niveluri de design.

Nivel 1: Designul general al sistemului

Aplicarea principiului „black box”: utilizatorul introduce date (adrese și configurări), iar sistemul simulează comportamentul memoriei cache și afișează rezultatele.

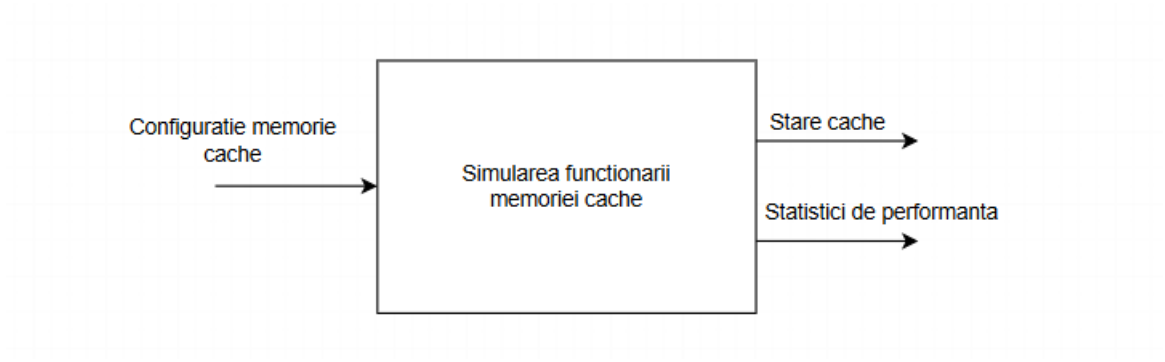


Figura 5 :Black Box al aplicatiei

Nivel 2:Divizarea in subsisteme

Arhitectura aplicației *Simularea unei memorii cache* poate fi împărțită în trei subsisteme principale, fiecare cu roluri bine definite în funcționarea aplicației**.**

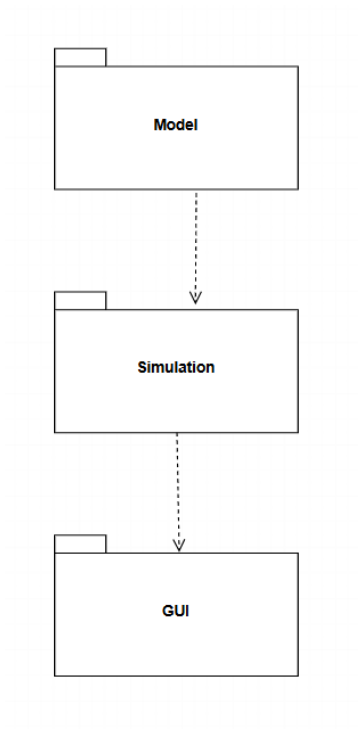


Figura 6: Diagrama de pachete a aplicatiei

1. Model

Conține implementările structurilor de date necesare simulării memoriei cache și ale memoriei principale. Include doar logica de stocare, fără interacțiune cu utilizatorul sau decizie asupra politicilor de înlocuire. Oferă suportul de bază pentru operațiile de citire, scriere și validare a blocurilor din cache.

2. Simulation

Pachet responsabil de logica de control a simulării. Acesta gestionează comportamentul cache-ului în timpul acceselor la memorie. Aici sunt implementate politicile de înlocuire (FIFO,LRU în cazul nostru), determinarea rezultatelor HIT/MISS, încărcarea blocurilor din memoria principală și calculul statisticilor.

3. GUI

Pachet dedicat interfeței grafice și interacțiunii cu utilizatorul. Acesta preia adresele introduse, transmite cererea către subsistemul de simulare și afișează vizual rezultatele. Oferă o reprezentare grafică a memoriei cache (stare HIT/MISS, culori, valori) și funcționalități de control precum pornirea, oprirea și resetarea simulării.

Nivel 3.Divizarea in clase

Clasele din proiect sunt interconectate conform unei arhitecturi modulare, în care logica simulării este separată de structurile de date care modelează memoria cache și memoria principală.

Componenta centrală a simulării este reprezentată de clasa CacheSimulator, care gestionează accesările memoriei cache, determinând dacă acestea se finalizează cu HIT sau MISS. CacheSimulator depende de clasele FifoCache,LruCache și MainMemory pentru operarea efectivă a simulației: dacă blocul solicitat nu este găsit în cache (MISS), datele sunt extrase din MainMemory, iar FifoCache si LruCache decide unde va fi plasat blocul în structura cache-ului conform politicii de inlocuire alese.

Clasa FifoCache extinde clasa Cache, moștenind toate operațiile generice pentru manipularea liniilor cache și adăugând algoritmul FIFO pentru selectarea liniei victime atunci când cache-ul este plin. Această moștenire asigură reutilizarea codului și posibilitatea de a integra ulterior alte politici de înlocuire fără a modifica clasa Cache.

Ulterior a fost adaugata clase LruCache ,care a fost implementata extrem de usor,extinzand de asemenea clasa Cache asemenea clasei FifoCache.

Clasa Cache este responsabilă de reprezentarea structurii memoriei cache și realizează o compoziție asupra unui vector de obiecte CacheBlock. Fiecare CacheBlock reprezintă o linie din cache care conține un tag, datele blocului și un bit de validare.

Compoziția garantează că liniile din cache există doar ca parte a obiectului Cache, reflectând fidel organizarea fizică a memoriei cache la nivel hardware.

Clasa MainMemory simulează memoria principală și furnizează datele necesare încărcării blocurilor în cache. Interacțiunea dintre CacheSimulator și MainMemory este una de dependență funcțională, deoarece simulatorul apelează metodele de citire din memoria principală pentru a trata MISS-urile.

Prin această organizare, proiectul permite o separare clară între logica de simulare (Control) și structurile de date (Model), făcând sistemul ușor de extins și întreținut.

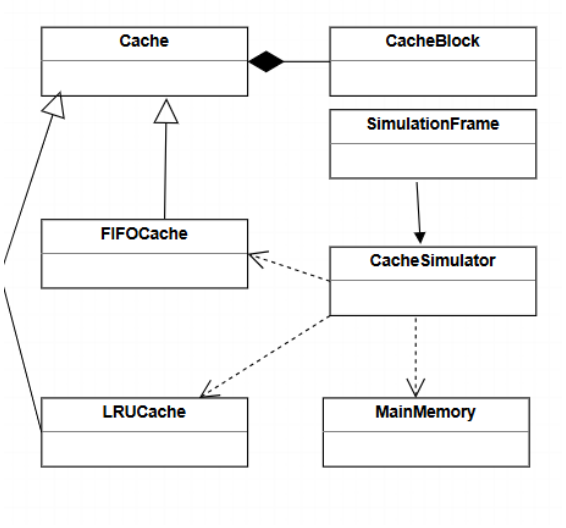


Figura 7:Relatiile dintre clasele aplicatiei

SimulationFrame și CacheSimulator colaborează permanent pentru a asigura funcționarea corectă a aplicației,avand o relatie de asociere uni-directionala.

Fereastra principală preia acțiunile utilizatorului — cum ar fi accesarea sau scrierea unei adrese — și le transmite către clasa CacheSimulator, care conține logica principală a simulării.

Comunicarea dintre cele două componente se realizează prin apelul metodelor:

- access(int address) – care gestionează operațiile de citire din memorie
- writeData(int address, int value) – tratează operațiile de scriere, conform politicii Write-Through, și returnează rezultatul operației.

După fiecare apel, simulatorul transmite interfeței rezultatul, iar aceasta actualizează automat elementele vizuale: culoarea liniilor din cache (verde pentru HIT, roșu pentru MISS, galben pentru scriere) și valorile statisticilor.

Astfel, interfața grafică și simulatorul formează un mecanism unitar de procesare și afișare, în care fiecare operație este reflectată vizual în timp real.

6.2 Interfata grafica

Interfața grafică reprezintă modul principal prin care utilizatorul interacționează cu aplicația.

În cadrul proiectului, am implementat o interfață simplă și intuitivă, bazată pe biblioteca Swing, care permite configurarea rapidă a simulării și observarea în timp real a modului în care funcționează memoria cache.

Fereastra principală a aplicației este gestionată de clasa SimulationFrame, care extinde JFrame și integrează toate elementele grafice.

Structura interfeței este împărțită în mai multe zone funcționale:

- partea superioară conține un singur buton „Panou de control”, care deschide o fereastră secundară dedicată configurării și controlului simulării;
- zona centrală afișează vizual liniile cache-ului, actualizate dinamic în funcție de operațiile efectuate;
- partea de jos afișează memoria principală, sub formă de grilă.

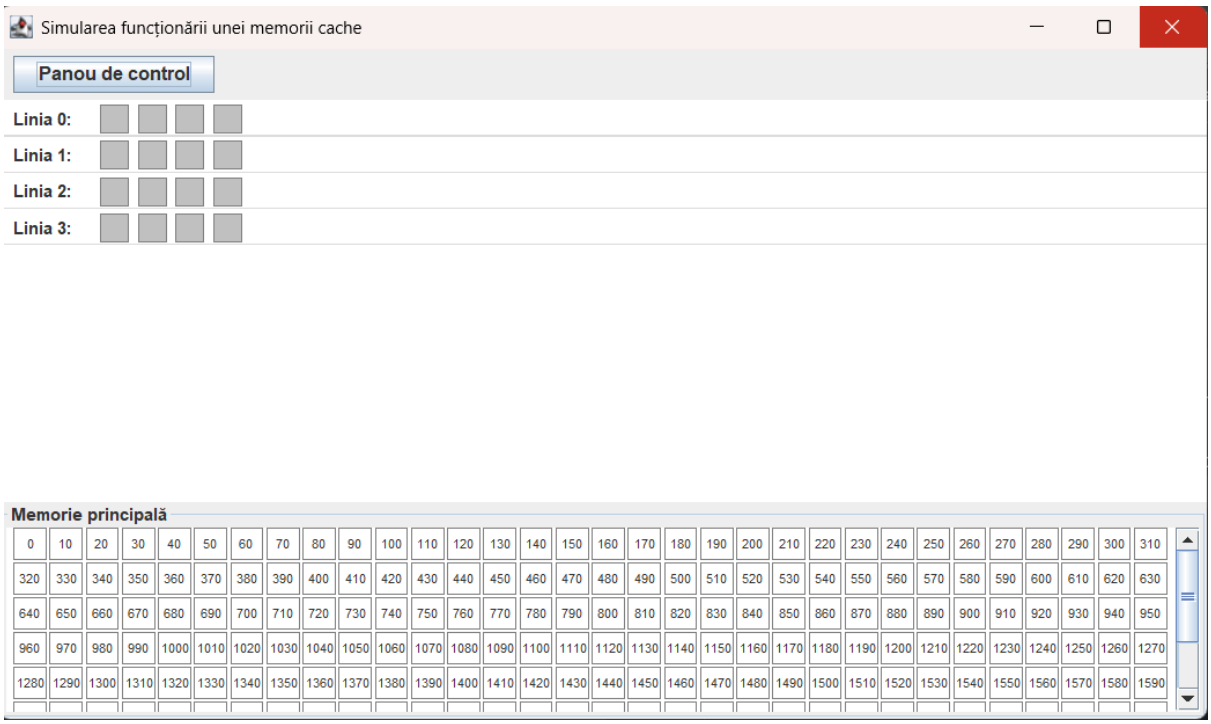


Figura 8: Fereastra principala a aplicatiei

În fereastra de panou de control, utilizatorul are la dispoziție câmpuri de introducere pentru adresă și valoare, butoane pentru accesarea sau scrierea unei adrese, butonul de resetare a cache-ului,posibilitatea alegerii tipului de politica de

inlocuire pe care doreste sa simuleze datele din fisier precum și opțiunea de vizualizare statisticilor.

Tot de aici se poate porni și simularea automată pe baza unui fișier de operații (operatii.txt), care conține instrucțiunile de citire și scriere.

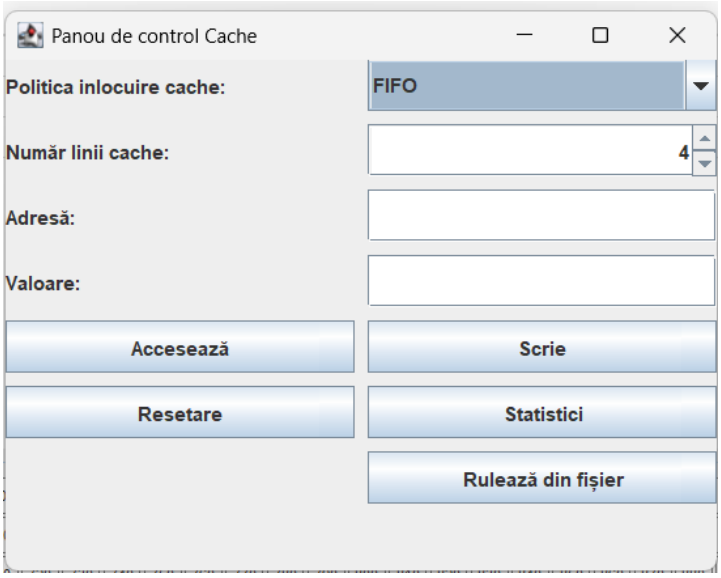


Figura 9: Panoul de control al aplicatiei

Statisticile sunt afișate într-o fereastră separată, unde se pot vizualiza numărul total de HIT-uri și MISS-uri, precum și procentajele corespunzătoare, reprezentate grafic prin bare de progres (JProgressBar).

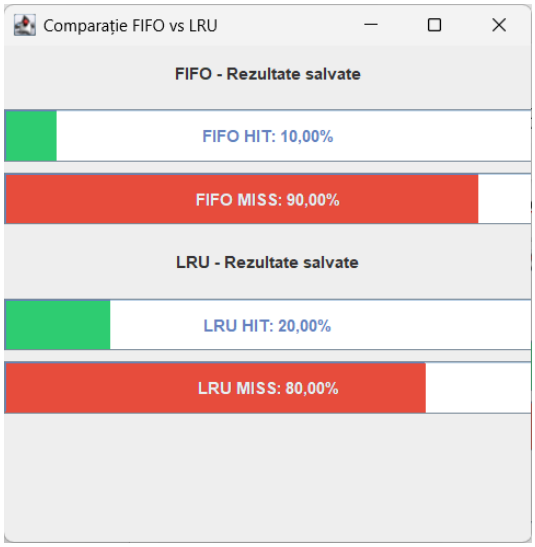


Figura 10:Fereastra pentru afisarea statisticilor

Prin această abordare, interfața nu este doar un instrument de control, ci și unul educativ, care permite înțelegerea intuitivă a comportamentului unei memorii cache reale.

Codul este structurat clar, cu secțiuni separate pentru configurare, actualizare grafică și gestionarea interacțiunii cu utilizatorul.

7.Implementare

Această secțiune descrie structura internă a aplicației, modul de implementare a componentelor software, politicile de înlocuire utilizate și organizarea sistemului grafic. În versiunea finală, aplicația suportă două politici de înlocuire – **FIFO** și **LRU** – care pot fi selectate de utilizator din interfață. De asemenea, aplicația permite salvarea separată a rezultatelor pentru fiecare politică, facilitând astfel compararea performanțelor.

7.1 Librarii utilizate

Pentru realizarea aplicației am folosit exclusiv biblioteci standard din limbajul **Java**, fără dependențe externe. Acestea oferă suport pentru interfața grafică, manipularea datelor și gestionarea evenimentelor.

Principalele librării utilizate sunt:

- **javax.swing** – folosită pentru realizarea interfeței grafice a aplicației.Biblioteca Swing permite crearea de ferestre, panouri, butoane, câmpuri de text și alte componente vizuale.Ea a fost utilizată pentru fereastra principală (JFrame), panourile grafice (JPanel), butoane (JButton), câmpuri de introducere (JTextField) și componente suplimentare precum JProgressBar, JSpinner sau JScrollPane.Swing a fost aleasă datorită simplității și a faptului că face parte din standardul Java, fiind potrivită pentru aplicații educaționale.

- **java.awt** – utilizată pentru gestionarea elementelor grafice de nivel mai jos, cum ar fi culorile, fonturile și layout-urile.Pachetul AWT (Abstract Window Toolkit) a fost folosit pentru organizarea componentelor în fereastră (BorderLayout, FlowLayout) și pentru setarea culorilor (de exemplu, evidențierea HIT-urilor și MISS-urilor prin verde sau roșu).

- **java.util** – oferă clase și colecții de bază folosite în program, cum ar fi Timer, ArrayList sau Random.În proiect, clasa Timer a fost folosită pentru animarea culorilor blocurilor din cache (culoarea revine la alb după o perioadă scurtă de timp).

- **java.io** – utilizată pentru citirea și scrierea datelor din fișiere. Acest pachet permite citirea automată a operațiilor din fișierul operatii.txt și salvarea rezultatelor în fișierul rezultate.txt.Astfel, aplicația poate rula simulări fără intervenția utilizatorului și poate genera un raport complet al execuției.

7.2 Clasele aplicatiei

Aplicația este formată din mai multe clase care colaborează pentru a simula funcționarea unei memorii cache.

Pentru o înțelegere mai clară, acestea vor fi prezentate începând cu cele mai simple structuri de date și terminând cu interfața grafică, care le folosește pentru a afişa rezultatele.

5.2.1 Clasa CacheBlock

Această clasă reprezintă cea mai mică unitate logică a cache-ului – o linie de memorie.

Fiecare bloc conține:

- un **tag**, care identifică blocul în memorie;
- un **bit de validitate**, care indică dacă blocul conține date corecte;
- un **vector de valori**, care reține conținutul efectiv al blocului.

Metode principale :

- isValid() – verifică dacă blocul este valid;
- invalidate() – marchează blocul ca nevalid, atunci când este înlocuit.

5.2.2 Clasa Cache

Clasa Cache reprezintă structura de bază a memoriei cache și este clasa părinte pentru clasa FifoCache.

Principalul atribut al acestei clase este:

- protected CacheBlock[] lines – vectorul de linii din cache, fiecare reprezentând un bloc de memorie;

Metode principale :

- **find(int tag)** – caută o linie care conține un anumit tag și returnează indexul ei; dacă nu o găsește, întoarce -1 (MISS);
- **write(int index, int tag, int[] blockData)** – scrie un bloc de date într-o linie specificată și o marchează ca validă;
- **invalidate(int index)** – invalidează o linie din cache, marcând-o ca neutilizabilă;
- **size()** – returnează numărul total de linii din cache;
- **getLines()** – permite accesul la vectorul de linii pentru alte clase (cum ar fi simulatorul).

5.2.3 Clasa FifoCache

Clasa FifoCache extinde clasa de bază Cache și implementează politica de înlocuire FIFO (First In, First Out), adică „primul intrat – primul ieșit”.

Această politică stabilește care bloc va fi inlocuit atunci când memoria cache este plină.

Clasa folosește o coadă (Queue) pentru a reține ordinea în care liniile din cache au fost utilizate, astfel încât să poată identifica rapid care este cea mai veche.

Metode principale:

- **selectVictim()** – alege linia care urmează să fie înlocuită.
  Dacă există o linie invalidă (nefolosită), aceasta este selectată automat.
  În caz contrar, metoda elimină din coadă prima linie introdusă (cea mai veche) și o returnează drept victimă.
- **afterWrite(int index)** – actualizează coada după o operație de scriere.Mai întâi elimină linia dacă se află deja în coadă (pentru a evita duplicatele), apoi o adaugă la final, marcând-o drept cea mai recent utilizată.

5.2.4 Clasa LruCache

Clasa LruCache implementează politica de înlocuire *Least Recently Used (LRU)* și extinde clasa de bază Cache. Ea este responsabilă de gestionarea ordinii de utilizare a liniilor cache, astfel încât, atunci când este necesară înlocuirea unui bloc, să fie selectată linia folosită cel mai puțin recent.

Atribute principale:

- **List<Integer> usageList** – listă care păstrează ordinea de utilizare a liniilor cache.Ultimul element din listă reprezintă *cea mai recent folosită* linie, iar primul element reprezintă cea mai puțin recent folosită.

Metode principale:

- **public void updateUsage(int index)-**Această metodă actualizează ordinea liniilor cache după o accesare sau o scriere:elimină indexul dacă acesta există deja în listă;adaugă indexul la finalul listei (devine *cea mai recent folosită* linie).

Aceasta este operația fundamentală care menține corect politica LRU.

- **public void invalidate(int index)-**Această metodă invalidează manual o linie: marchează linia ca nevalidă prin super.invalidate(index),elimină indexul din lista LRU (usageList.remove(index)).

Eliminarea este necesară pentru a menține consistența structurii de date LRU.

- **public void write(int index, int tag, int[] blockData)-**Metoda suprascrie comportamentul clasei Cache:setează blocul în linia respectivă (apelând super.write()),actualizează ordinea LRU prin updateUsage(index).

Scrierea unui bloc îl marchează ca *recent folosit*, conform comportamentului LRU.

5.2.5 Clasa MainMemory

Clasa MainMemory are rolul de a simula **memoria principală** a sistemului.
Ea reprezintă locul unde sunt stocate toate datele, iar cache-ul preia sau scrie informații din și către această zonă.
În cadrul aplicației, MainMemory funcționează ca o memorie RAM simplificată, implementată sub forma unui tablou de valori întregi.

Principalul atribut este:

- private int[] mem – vectorul care stochează valorile din memoria principală. Fiecare element al acestui tablou reprezintă o celulă de memorie, identificată printr-o adresă numerică.Dimensiunea tabloului este stabilită în momentul creării obiectului și este transmisă prin constructor.

Metode principale :

- **public int read(int address)** – citește și returnează valoarea aflată la adresa specificată.Se folosește expresia address % mem.length pentru a preveni erorile de indexare, simulând astfel un sistem de adresare circulară.
- **public void write(int address, int value)** – scrie o valoare la adresa dată. Este utilizată în cadrul politicii **Write-Through**, unde orice modificare efectuată în cache este imediat reflectată și în memoria principală, pentru a asigura consistența datelor.

5.2.6 Clasa CacheSimulator

Clasa CacheSimulator reprezintă componenta logică centrală a aplicației și este responsabilă de simularea completă a funcționării memoriei cache. Ea gestionează operațiile de citire, scriere și eliminare, aplicând una dintre politicile de înlocuire selectate de utilizator: FIFO sau LRU. De asemenea, clasa se ocupă de calculul statisticilor de performanță și de salvarea acestora în fișiere separate, pentru compararea rezultatelor.

Atribute principale:

- private FifoCache cache**;** – obiect care gestionează liniile cache-ului și politica de înlocuire **FIFO**;
- private MainMemory memory; – reprezintă memoria principală a sistemului, din care se citesc sau în care se scriu date;
- private int hits; – contorizează numărul de accesări reușite (**HIT-uri**);

- private int misses**;** – contorizează numărul de accesări ratate (**MISS-uri**);
- private PrintWriter logWriter; – permite salvarea în fișier a rezultatelor simulării, pentru analiză ulterioară.

  Metode principale:

- **public boolean access(int address)-** Realizează o operație de citire din cache. Metoda calculează tag-ul și offset-ul corespunzător adresei și caută blocul în cache prin apelul cache.find(tag).

➢ Dacă blocul este găsit (HIT), metoda incrementează contorul HIT și afișează un mesaj corespunzător. Politica de înlocuire este actualizată automat: în cazul LRU se marchează linia ca fiind recent utilizată, iar în cazul FIFO ordinea rămâne neschimbată.

➢ Dacă blocul nu este găsit (MISS), metoda incrementează contorul MISS, citește blocul din memoria principală folosind memory.read(), selectează o linie victimă în funcție de politica activă (selectVictim() din FifoCache sau LruCache) și încarcă noul bloc în cache. În cazul politicii FIFO este apelată suplimentar metoda afterWrite() pentru actualizarea ordinii interne.

- **public boolean writeData(int address, int value)-** Această metodă simulează o operație de scriere utilizând politica Write-Through, în care orice actualizare efectuată în cache este propagată imediat în memoria principală. Metoda determină tag-ul și offset-ul adresei și verifică dacă blocul se află în cache.

➢ În cazul unui WRITE HIT, valoarea este actualizată în blocul existent, scrisă imediat și în memoria principală, iar politica de înlocuire este actualizată (LRU marchează linia ca recent utilizată, FIFO nu modifică ordinea).

➢ În cazul unui WRITE MISS, blocul este încărcat din memoria principală, se selectează o victimă în funcție de politica activă (FIFO sau LRU), blocul este plasat în cache, iar apoi valoarea este scrisă atât în cache, cât și în memoria principală conform politicii Write-Through.

  Metoda returnează true pentru WRITE HIT și false pentru WRITE MISS.

- **public void evict(int address)-** Această metodă permite invalidarea manuală a unei linii din cache, simulând eliminarea forțată a unui bloc. Metoda calculează tag-ul asociat adresei și verifică dacă blocul corespunzător se află în cache, utilizând cache.find(tag).

➢ Dacă blocul este găsit, metoda:

➢ invalidează linia respectivă prin apelul invalidate(index), operație care marchează blocul ca nevalid;

- în cazul politicii LRU, invalidarea elimină linia și din structura internă usageList, asigurând funcționarea corectă a algoritmului LRU după eliminare;
- în cazul politicii FIFO, linia este pur și simplu marcată ca nevalidă, ordinea în coada FIFO rămânând neschimbată.

Metoda nu încarcă automat un bloc nou după invalidare; ea doar elimină blocul selectat, oferind utilizatorului control manual asupra structurii cache-ului.

- **public void printCacheState()-**afișează în consolă (și în fișierul de log) starea completă a cache-ului.Pentru fiecare linie, metoda utilizează informațiile din CacheBlock – precum tag-ul, starea de validitate și valorile blocului – pentru a construi o imagine clară a conținutului memoriei cache.
- **public void printMainMemory(int start, int end)-**afișează un fragment din memoria principală, apelând metoda **read()** din MainMemory pentru fiecare adresă din intervalul specificat.Este utilă pentru verificarea faptului că politica Write-Through a fost aplicată corect.
- **public void startLogging(String fileName) și public void stopLogging()-** gestionează procesul de salvare a rezultatelor simulării într-un fișier text. Prima metodă creează un fișier specific politicii de înlocuire folosite (FIFO sau LRU), deschide fluxul de scriere și adaugă antetul simulării. Metoda complementară, stopLogging(), închide logarea: inserează în fișier statisticile finale ale execuției – numărul total de HIT-uri, MISS-uri și rata de succes – apoi închide în siguranță fluxul de ieșire. Astfel, simularea oferă un mecanism simplu și automatizat de generare a rapoartelor pentru fiecare politică de cache.

```
=== SIMULARE CACHE ===
Politică: FIFO

MISS (FIFO) - pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ
```

Figura 11 **:**Exemplu de fisier pentru feedback

Metode auxiliare:

- **getHits()**, **getMisses()**, **getHitRate()**, **getMissRate()** – calculează și returnează valorile statistice folosite ulterior în interfața grafică (SimulationFrame);
- **getMemoryValue(int address)** – permite interfeței să acceseze direct valorile din memoria principală, apelând metoda read() din MainMemory.

5.2.7 SimulationFrame

Clasa SimulationFrame reprezintă interfața grafică principală a aplicației și este responsabilă de interacțiunea utilizatorului cu aplicatia.

Metode principale:

- **openControlPanel()** – creează și afișează o fereastră separată cu butoane și câmpuri de control pentru interacțiune (acces, scriere, resetare, statistici, rulare din fișier).
- **runFromFile(String filePath)** – citește secvențial operațiile din fișierul text *operatii.txt* și le execută automat.
  Această metodă apelează metodele access() și writeData() din CacheSimulator pentru fiecare linie din fișier.

```
R 0
R 4
R 8
R 0
E 4
R 4
```

Figura 12 :Exemplu de continut pentru fisierul operatii.txt

- **updateCacheView()** – actualizează conținutul vizual al panoului cache în funcție de starea actuală a simulatorului.
  Utilizează informațiile din CacheBlock[] obținute prin simulator.getCacheLines().
- **updateMainMemoryView()** – reîmprospătează panoul care afișează memoria principală, citind datele din CacheSimulator prin getMemoryValue().
- **colorSquare() / colorMainMemoryCell()** – realizează colorarea temporară a celulelor din cache și memorie pentru a evidenția operațiile de citire/scriere.
- **showStatsWindow()** – afișează o fereastră cu statisticile actuale (număr de HIT-uri și MISS-uri, rate procentuale), inclusiv două bare de progres (JProgressBar) care ilustrează vizual performanța cache-ului.

7.3 Arhitectura aplicatiei

După cum am evidențiat în cadrul capitolului dedicat designului, la Nivelul 2 – Divizarea în subsisteme, arhitectura aplicației este împărțită în trei componente majore: interfața grafică, componenta de simulare și structurile de memorie – care colaborează pentru a reproduce comportamentul unei memorii cache reale. Cele trei niveluri sunt implementate în pachete separate, ceea ce permite o organizare clară și o separare a responsabilităților, facilitând extinderea și întreținerea proiectului.

La nivel superior se află interfața grafică (pachetul *GUI*), reprezentată de clasa SimulationFrame. Aceasta este responsabilă atât de afișarea stării curente a cache-ului și memoriei principale, cât și de preluarea comenzilor utilizatorului, precum accesarea unei adrese, scrierea unei valori, resetarea cache-ului sau încărcarea operațiilor dintr-un fișier. Evenimentele generate în interfață sunt transmise direct componentei de simulare, fără ca GUI-ul să gestioneze logică de calcul sau algoritmică internă.

Nivelul intermediar este reprezentat de clasa CacheSimulator, componenta centrală a aplicației. Aceasta coordonează întregul proces de simulare, realizând legătura dintre operațiile utilizatorului și mecanismele interne ale memoriei cache. Simulatorul determină tag-ul, offset-ul și blocul asociat fiecărei adrese, identifică dacă blocul se află în cache, actualizează statisticile de performanță și aplică politica de înlocuire selectată (FIFO sau LRU). În acest nivel se află și logica pentru scrierea în fișierele de rezultate, ceea ce permite analizarea comparativă a celor două politici.

Cel de-al treilea nivel îl reprezintă structurile interne care modelează efectiv memoria cache și memoria principală (pachetul *Model*). Clasa Cache furnizează o structură comună pentru gestionarea liniilor cache-ului, iar clasele FifoCache și LruCache extind această structură și implementează comportamente specifice politicilor de înlocuire. Fiecare linie a cache-ului este reprezentată printr-un obiect CacheBlock, care conține tag-ul, starea de validitate și datele blocului. Memoria principală, implementată în clasa MainMemory, simulează un spațiu liniar de adrese și reprezintă sursa reală a datelor utilizate în procesul de încărcare a blocurilor în cache.

Interacțiunea dintre cele trei niveluri este una directă și firească: GUI transmite operațiile către CacheSimulator, simulatorul aplică logica algoritmică și actualizează structurile din pachetul *Model*, iar rezultatele sunt returnate în interfață sub formă vizuală. Această separare clară asigură modularitate, permite modificarea politicilor de înlocuire fără a afecta GUI-ul și oferă flexibilitatea necesară extinderii aplicației cu algoritmi suplimentari sau noi moduri de mapare

7.4 Algoritmi utilizati

Acest subcapitol prezintă în detaliu algoritmii implementați pentru gestionarea memoriei cache, respectiv politicile de **mapare**, **înlocuire** și **scriere**. Fiecare algoritm influențează modul în care datele sunt aduse, păstrate și eliminate din cache, afectând direct performanța sistemului (HIT-uri și MISS-uri).

7.4.1 Algoritmul FIFO

Politica de înlocuire FIFO (First-In, First-Out) este una dintre cele mai simple și intuitive metode de gestionare a liniilor dintr-un cache. Principiul de bază constă în eliminarea blocului care se află de cel mai mult timp în memorie, fără a ține cont de recența sau frecvența acceselor. În acest model, cache-ul se comportă precum o coadă: primul bloc încărcat este și primul candidat pentru înlocuire.

În cadrul implementării folosite, comportamentul FIFO este realizat printr-o structură de tip coadă (Queue<Integer>), în care sunt păstrați indicii liniilor în ordinea în care acestea devin valide. Atunci când un bloc este adus în cache pentru prima dată, indexul liniei respective este introdus în această coadă.

Mecanismul de înlocuire este gestionat prin funcția selectVictim(), care are două roluri majore:

1. **Căutarea unei linii invalide**
   Funcția parcurge toate liniile din cache și, dacă găsește una care nu este validă, o selectează pentru scriere și îi adaugă indexul în coadă. În acest caz, nu este nevoie de înlocuirea unui bloc existent.

2. **Selectarea unei victime atunci când cache-ul este plin**
   Dacă toate liniile sunt valide, funcția extrage primul index aflat în coadă, folosind operația fifoQueue.poll(). Acesta reprezintă blocul cel mai vechi aflat în cache și devine automat victima care va fi suprascrisă.

După ce o linie este actualizată cu un bloc nou, sistemul trebuie să mențină ordinea corectă a cozii FIFO. Această responsabilitate revine funcției afterWrite(int index), care reintroduce linia actualizată în coadă:

- mai întâi elimină indexul liniei din coadă dacă acesta există deja (fifoQueue.remove(index)), pentru a preveni duplicarea;
- apoi adaugă indexul la final (fifoQueue.offer(index)), marcând faptul că această linie este cea mai nou introdusă în cache.

Astfel, coada păstrează în permanență ordinea exactă a liniilor valide: cele introduse mai devreme vor fi eliminate primele, iar cele mai recente vor rămâne disponibile pentru o perioadă mai lungă.

Politica FIFO este ușor de implementat și nu necesită urmărirea istoricului de accesare a blocurilor, ceea ce o face eficientă din punct de vedere al costului de execuție. Totuși, o limitare importantă este faptul că poate elimina un bloc încă util, doar pentru că acesta a fost introdus înaintea altora. Cu toate acestea, simplitatea și predictibilitatea ei o recomandă în multe scenarii de simulare sau modelare a comportamentului memoriei cache.

7.4.2 Algoritmul LRU

Politica de înlocuire LRU (Least Recently Used) are ca scop eliminarea blocului care a fost accesat cel mai puțin recent. Spre deosebire de FIFO, care ține cont doar de ordinea de sosire a blocurilor în cache, LRU urmărește activ momentul în care fiecare linie a fost folosită ultima dată, încercând să păstreze cât mai mult blocurile „calde”, care sunt folosite frecvent.

În această implementare, ordinea utilizării este gestionată printr-o listă (List<Integer> usageList>), care menține indicii liniilor cache în funcție de recența acceselor. Ultimul index din listă este întotdeauna cel mai recent utilizat, iar primul este cel mai vechi, deci candidatul natural pentru eliminare.

Actualizarea ordinii de utilizare este realizată prin funcția updateUsage(int index), care elimină indexul liniei din lista curentă și îl adaugă la final, marcând faptul că linia a fost accesată recent. Această funcție este apelată în două momente importante:

1. **Când un bloc este găsit în cache**
   Funcția find(int tag) caută o linie validă având tag-ul corespunzător. Dacă aceasta este găsită, metoda apelează updateUsage(i) pentru a marca linia drept recent accesată.

2. **Când un bloc este scris în cache**
   Funcția write(int index, int tag, int[] blockData), care suprascrie conținutul unei linii, apelează de asemenea updateUsage(index) pentru a actualiza ordinea recenței.

De asemenea, atunci când o linie devine invalidă, funcția invalidate(int index) elimină din listă indexul acelei linii, pentru a evita păstrarea unor poziții care nu mai sunt relevante în mecanismul LRU.

Selecția victimei este realizată de funcția selectVictim(), care operează în două etape:

1. **Identificarea liniilor încă neutilizate**
   Dacă lista de utilizări nu acoperă toate liniile cache-ului (este mai scurtă decât numărul total de linii), funcția caută o linie care nu apare încă în listă. Aceasta este o linie nouă, încă neutilizată, și poate fi folosită direct pentru înlocuire.
2. **Selectarea liniei cel mai puțin recent folosite**
   Dacă toate liniile au fost folosite, victima este pur și simplu primul element din listă, eliminat prin usageList.remove(0). Aceasta reprezintă linia accesată cel mai de mult timp, deci exact definiția LRU.

Prin această structură, lista usageList reflectă în permanență istoricul real al acceselor în cache: liniile recent folosite sunt mutate la final, în timp ce cele vechi rămân în față. Astfel, politica LRU oferă un comportament adaptiv, potrivit situațiilor în care accesul la date urmează modele locale sau repetitive.

7.4.3 Politica de scriere Write-Through

Simularea folosește politica de scriere write-through, ceea ce înseamnă că orice operație de scriere efectuată asupra unei linii de cache este propagată imediat și în memoria principală, fără întârzieri și fără a marca linia drept „murdară”. În cod, acest comportament este vizibil în metodele de scriere: atât în situația unui WRITE HIT, cât și în cazul unui WRITE MISS, valoarea este actualizată în cache și transmisă direct către memoria principală:

- actualizarea liniei din cache
  cache.getLines()[index].getBlock()[offset] = value;
- actualizarea imediată în memoria principală
  memory.write(address, value);

Această abordare elimină necesitatea unui „dirty bit” și simplifică logica de evictare, deoarece blocurile din cache sunt întotdeauna sincronizate cu memoria principală. Deși acest model implică un număr mai mare de accesări ale memoriei, el garantează coerență totală între cache și memorie în orice moment.

7.4.4 Structura de tip Fully-Associative

Cache-ul implementat este de tip fully-associative, ceea ce înseamnă că orice bloc din memoria principală poate fi plasat în oricare dintre liniile cache-ului. Această flexibilitate maximizează potențialul de reutilizare, deoarece nu există restricții privind poziția în care un anumit bloc poate fi stocat. Singura operație obligatorie este căutarea unui bloc după tag, care trebuie realizată parcurgând toate liniile cache-ului. Deși această metodă poate fi, teoretic, mai costisitoare decât mapările directe, ea este

potrivită pentru un simulator educațional și permite aplicarea corectă a politicilor FIFO și LRU fără alte constrângeri structurale.

7.5 Operatii fundamentale asupra memorie cache

În funcționarea unui cache apar trei operații esențiale: regăsirea unui bloc ,plasarea unui bloc în cache în urma unui miss ,și descărcarea unui bloc existent atunci când este necesară eliberarea unei linii .Simularea realizată implementează aceste operații în mod direct în cadrul metodelor de acces și scriere, iar comportamentul este identic pentru politicile FIFO și LRU, diferențele apărând doar în modul de selectare a victimei.

7.5.1 Regasirea(operatia de cautare in cache)

Regăsirea reprezintă primul pas în accesarea memoriei cache și presupune identificarea liniei care conține blocul cerut de procesor. Pentru fiecare adresă accesată, simulatorul extrage tag-ul asociat și verifică dacă acesta se află într-una dintre liniile cache-ului.

Implementarea este complet realizată în metoda find(), care parcurge toate liniile disponibile (specific structurii fully-associative). Pentru fiecare linie se verifică două condiții:

1. dacă linia este validă;
2. dacă tag-ul stocat în linie coincide cu tag-ul căutat.

Dacă ambele condiții sunt îndeplinite, metoda returnează indexul liniei, ceea ce semnifică un HIT. În caz contrar, metoda întoarce valoarea -1, indicând un MISS.

În cazul politicii LRU, o regăsire reușită are un efect suplimentar: metoda updateUsage(index) este apelată pentru a marca linia respectivă drept cea mai recent utilizată. Lista de utilizare (usageList) este actualizată în consecință, elementul fiind mutat la finalul listei. În schimb, politica FIFO nu modifică ordinea liniilor în timpul regăsirilor, întrucât această politică se bazează exclusiv pe ordinea în care blocurile au fost încărcate în cache, nu pe modul în care acestea sunt accesate ulterior.

7.5.2 Plasarea (operatia de incarcare in cache)

Plasarea unui bloc reprezintă procesul prin care un bloc din memoria principală este adus în cache în urma unui MISS. Această operație implică extragerea blocului corespunzător adresei accesate, selectarea unei linii disponibile sau a unei victime, și încărcarea blocului în linia respectivă.

În implementarea proiectului, plasarea are loc în metodele access() (pentru citire) și writeData() (pentru scriere), imediat după detectarea unui MISS. Mai întâi,

simulatorul calculează adresa de bază a blocului și îl încarcă din memoria principală într-un buffer temporar. Ulterior, este selectată o linie în care va fi plasat blocul.

Selecția victimei depinde de politica de înlocuire activă:

- pentru FIFO, metoda selectVictim() returnează fie o linie încă nefolosită, fie cea mai veche linie validă, determinată prin structura de tip coadă;
- pentru LRU, selectVictim() alege linia cel mai puțin recent utilizată, aceasta fiind primul element din lista de utilizare.

După determinarea victimei, blocul este scris în cache prin apelul metodei write() din clasa corespunzătoare (FifoCache sau LruCache). La final, în cazul LRU ordinea utilizării este actualizată automat în cadrul metodei write(), iar în cazul FIFO linia este adăugată în coadă prin apelul afterWrite() pentru a menține ordinea corectă a introducerii blocurilor.

7.5.3 Descarcarea (operatia de invalidare a unei linii din cache)

Descărcarea reprezintă procesul prin care o linie existentă în cache este eliberată pentru a face loc unui nou bloc. În simularea realizată, descărcarea este implicit integrată în logica metodelor selectVictim() ale celor două politici FIFO și LRU.

Atunci când cache-ul este plin și are loc un MISS, metoda selectVictim() întoarce indexul liniei care urmează să fie suprascris:

- în FIFO, coada internă reține ordinea în care liniile au devenit valide; poll() extrage linia cea mai veche, indicând astfel blocul care trebuie descărcat;
- în LRU, lista ordonează liniile după recența utilizării; remove(0) selectează linia accesată cel mai de mult timp, care reprezintă victima ideală.

În plus față de descărcarea automată determinată de aceste politici, simulatorul oferă și metoda evict(), care permite invalidarea unei linii pe baza adresei. Implementarea apelează find() pentru identificare, iar în caz de succes utilizează invalidate(index) pentru a marca linia ca invalidă și a o elimina din structurile interne (lista LRU sau coada FIFO).

Întrucât este folosită politica write-through, nu este necesară nicio operație suplimentară în timpul descărcării, deoarece datele sunt deja sincronizate cu memoria principală în momentul înlocuirii.

8. Testare

Scopul acestui capitol este validarea corectitudinii funcționării aplicației de simulare a memoriei cache și analiza comportamentului acesteia în diferite scenarii de acces la memorie. Testarea urmărește atât verificarea operațiilor de bază — citire și

scriere — cât și evaluarea impactului politicilor de înlocuire FIFO și LRU asupra performanței memoriei cache, măsurată prin ratele de HIT și MISS.

În urma rulării simulării, aplicația generează câte un fișier de rezultate pentru fiecare politică de înlocuire:

- rezultate_fifo.txt
- rezultate_lru.txt

Aceste fișiere conțin log-ul complet al execuției, incluzând fiecare operație efectuată, rezultatul acesteia (HIT sau MISS), starea curentă a cache-ului după fiecare accesare și statisticile finale.

8.1 Metodologia de testare

Testarea a fost realizată prin rularea simulatorului pe mai multe fișiere de operații, fiecare fișier fiind conceput pentru a evidenția un anumit tip de acces la memorie. Pentru fiecare scenariu, simularea a fost executată folosind ambele politici de înlocuire implementate (FIFO și LRU), iar rezultatele au fost salvate în fișiere separate de tip log.

În cadrul fiecărei rulări, au fost analizate:

- rezultatul fiecărei operații (HIT sau MISS);
- starea internă a memoriei cache după execuția operațiilor;
- statisticile finale, respectiv numărul total de HIT-uri și MISS-uri, precum și ratele procentuale corespunzătoare.

Această abordare permite compararea directă a comportamentului cache-ului în funcție de tipul de operații și de politica de înlocuire utilizată.

8.2 Categorii de teste

8.2.1 Teste de citire

Testele de citire urmăresc comportamentul memoriei cache în cazul accesărilor de tip citire, fără modificarea conținutului memoriei principale. Aceste scenarii includ accesări repetate ale aceloraşi adrese, accesări secvențiale și accesări mixte, având ca scop evidențierea principiului de localitate temporală și observarea diferențelor dintre politicile FIFO și LRU.

În aceste teste, se verifică apariția HIT-urilor după încărcarea inițială a blocurilor în cache și modul în care politicile de înlocuire influențează păstrarea sau eliminarea blocurilor.

```
R 0
R 4
R 8
R 12
R 0
R 16
R 0
R 4
R 20
R 24
```

Figura 13 :Fisierul de operatii utilizat pentru acest tip de teste

8.2.1.1 Rezultate asteptate

Având în vedere configurația sistemului simulat — memorie cache de tip fully-associative, cu un număr limitat de linii și politică de scriere write-through — comportamentul inițial al aplicației este predictibil. La începutul simulării, cache-ul este gol, astfel că primele accesări ale adreselor de memorie vor genera inevitabil MISS-uri, deoarece datele nu sunt încă încărcate în cache.

Pe măsură ce blocurile sunt aduse din memoria principală și plasate în cache, accesările repetate la aceleași adrese ar trebui să conducă la apariția unor HIT-uri, evidențiind principiul de localitate temporală.

În ceea ce privește politicile de înlocuire analizate, se anticipează diferențe clare de performanță:

- Politica FIFO este așteptată să producă o rată de HIT mai scăzută, deoarece înlocuiește blocurile strict în funcție de ordinea de introducere, fără a ține cont de recența utilizării acestora. Astfel, blocuri încă frecvent accesate pot fi eliminate prematur.
- Politica LRU este așteptată să obțină o rată de HIT mai ridicată, deoarece păstrează în cache blocurile utilizate recent și elimină datele care nu au mai fost accesate de o perioadă mai lungă. Acest comportament este mai eficient în scenarii care prezintă localitate temporală.

Prin urmare, se estimează că politica LRU va avea performanțe superioare comparativ cu FIFO, reflectate printr-un număr mai mare de HIT-uri și un număr mai mic de MISS-uri.

8.2.1.2 Rezultatele obtinute pentru politica FIFO

În urma rulării simulării utilizând fișierul operatii.txt și politica de înlocuire FIFO, s-a observat un comportament conform cu așteptările teoretice. Cache-ul a înlocuit de

fiecare dată blocul cel mai vechi introdus, fără a lua în considerare accesările ulterioare.

Analiza fișierului rezultate_fifo.txt evidențiază următoarele aspecte:

- primele accesări generează exclusiv MISS-uri, deoarece cache-ul este inițial gol;
- apare un număr foarte redus de HIT-uri, doar în situațiile în care o adresă este accesată din nou înainte ca blocul corespunzător să fie eliminat;
- pe măsură ce cache-ul se umple, blocurile sunt eliminate rapid, chiar și în cazurile în care acestea ar putea fi reutilizate ulterior.

Aceste rezultate confirmă limitările politicii FIFO, care nu ține cont de comportamentul real al accesărilor și poate elimina date încă relevante pentru execuția aplicației.

```
=== SIMULARE CACHE ===
Politică: FIFO

MISS (FIFO) - pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 3

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | TAG=3 | Bloc: 120 130 140 150

HIT (FIFO) - linia 0
MISS (FIFO) - pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=4 | Bloc: 160 170 180 190
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | TAG=3 | Bloc: 120 130 140 150

MISS (FIFO) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=4 | Bloc: 160 170 180 190
Linia 1 | TAG=0 | Bloc: 0 10 20 30
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | TAG=3 | Bloc: 120 130 140 150

MISS (FIFO) - pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=4 | Bloc: 160 170 180 190
Linia 1 | TAG=0 | Bloc: 0 10 20 30
Linia 2 | TAG=1 | Bloc: 40 50 60 70
Linia 3 | TAG=3 | Bloc: 120 130 140 150

MISS (FIFO) - pus în linia 3

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=4 | Bloc: 160 170 180 190
Linia 1 | TAG=0 | Bloc: 0 10 20 30
Linia 2 | TAG=1 | Bloc: 40 50 60 70
Linia 3 | TAG=5 | Bloc: 200 210 220 230

MISS (FIFO) - pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=6 | Bloc: 240 250 260 270
Linia 1 | TAG=0 | Bloc: 0 10 20 30
Linia 2 | TAG=1 | Bloc: 40 50 60 70
Linia 3 | TAG=5 | Bloc: 200 210 220 230

HIT-uri: 1
MISS-uri: 9
Rată HIT: 10.00%
```

Figura 14 :Rezultatele obtinute pentru politica Fifo

8.2.1.3 Rezultatele obtinute pentru politica LRU

În cazul utilizării politicii LRU (Least Recently Used), rezultatele obținute indică o îmbunătățire clară a performanței cache-ului. Politica LRU reușește să păstreze în memorie blocurile accesate frecvent și să elimine acele blocuri care nu au mai fost utilizate recent.

Din analiza fișierului rezultate_lru.txt se pot observa următoarele:

- blocurile corespunzătoare adreselor accesate repetat sunt menținute mai mult timp în cache;
- numărul de HIT-uri este mai mare comparativ cu politica FIFO;
- anumite MISS-uri întâlnite în cazul FIFO sunt evitate datorită actualizării continue a ordinii de utilizare a liniilor cache.

Acest comportament demonstrează capacitatea politicii LRU de a se adapta dinamic la tiparul de accesare a memoriei.

```
=== SIMULARE CACHE ===
Politică: LRU

MISS (LRU) - pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (LRU) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (LRU) - pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ

MISS (LRU) - pus în linia 3

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | TAG=3 | Bloc: 120 130 140 150

HIT (LRU) - linia 0
MISS (LRU) - pus în linia 1
```

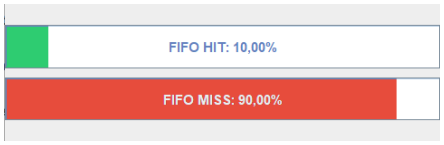
```
=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=4 | Bloc: 160 170 180 190
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | TAG=3 | Bloc: 120 130 140 150

HIT (LRU) - linia 0
MISS (LRU) - pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=4 | Bloc: 160 170 180 190
Linia 2 | TAG=1 | Bloc: 40 50 60 70
Linia 3 | TAG=3 | Bloc: 120 130 140 150

MISS (LRU) - pus în linia 3

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=4 | Bloc: 160 170 180 190
Linia 2 | TAG=1 | Bloc: 40 50 60 70
Linia 3 | TAG=5 | Bloc: 200 210 220 230

MISS (LRU) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=6 | Bloc: 240 250 260 270
Linia 2 | TAG=1 | Bloc: 40 50 60 70
Linia 3 | TAG=5 | Bloc: 200 210 220 230


HIT-uri: 2
MISS-uri: 8
Rată HIT: 20.00%
```

Figura 15:Rezultatele obtinute pentru politica Lru

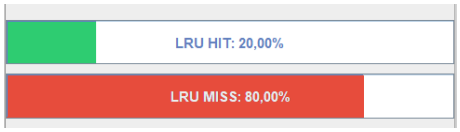
8.2.1.4 Interpretarea rezultatelor

Comparând rezultatele obținute, se observă că politica LRU oferă performanțe mai bune decât FIFO pentru acest set de date,asa cum ne-am si asteptat. Rata de HIT este dublă în cazul LRU, ceea ce demonstrează eficiența acestei politici în scenarii ce prezintă localitate temporală.

Fifo:



Lru:



Figurile 16 si 17: Statisticile pentru politicile Fifo si Lru

Rezultatele confirmă rolul important al politicii de înlocuire în optimizarea performanței memoriei cache și evidențiază avantajele utilizării unei politici adaptive, precum LRU, în detrimentul uneia simple, cum este FIFO.

8.2.2 Teste de scriere

Testele de scriere au fost realizate pentru a valida implementarea politicii de scriere write-through. Pentru fiecare operație de scriere, s-a verificat actualizarea simultană a valorii atât în memoria cache, cât și în memoria principală.

Aceste teste permit observarea diferenței dintre situațiile de WRITE HIT și WRITE MISS, precum și impactul operațiilor de scriere asupra ordinii interne a liniilor cache, în funcție de politica de înlocuire activă.

```
W 0 10
W 4 20
W 8 30
W 0 40
W 12 50
W 4 60
```

Figura 18:Fisierul pentru operatiile de scriere

8.2.2.1 Rezultate asteptate

Pentru testele de scriere, se așteaptă ca orice operație de scriere să conducă la actualizarea simultană a valorii atât în memoria cache, cât și în memoria principală, conform politicii write-through.

În cazul unui WRITE MISS, blocul corespunzător adresei scrise este încărcat în cache înainte de actualizare, iar în cazul unui WRITE HIT, valoarea este modificată direct în blocul existent. Similar testelor de citire, diferențele dintre FIFO și LRU sunt așteptate să apară doar în situațiile în care cache-ul este plin.

8.2.2.2 Rezulatate obtinute pentru politica Fifo

În urma rulării testelor de scriere folosind politica FIFO, s-a constatat că primele operații de scriere se finalizează cu WRITE MISS, blocurile fiind încărcate în cache din memoria principală. Scrierile ulterioare la aceleași adrese generează WRITE HIT-uri.

Valorile scrise sunt actualizate imediat în memoria principală, confirmând implementarea corectă a politicii write-through.

```
=== SIMULARE CACHE ===
Politică: FIFO

WRITE MISS (FIFO) - adresa 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | INVALIDĂ
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

Bloc TAG=0 -> pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 0 → valoare: 10

WRITE MISS (FIFO) - adresa 4

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

Bloc TAG=1 -> pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ
```

```
=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 4 → valoare: 20

WRITE MISS (FIFO) - adresa 8

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

Bloc TAG=2 -> pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | INVALIDĂ


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 8 → valoare: 30

WRITE HIT (FIFO) - linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | INVALIDĂ


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 0 → valoare: 40

WRITE MISS (FIFO) - adresa 12
```

```
WRITE MISS (FIFO) - adresa 12

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | INVALIDĂ

Bloc TAG=3 -> pus în linia 3

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | TAG=3 | Bloc: 50 130 140 150


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 12 → valoare: 50

WRITE HIT (FIFO) - linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 60 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | TAG=3 | Bloc: 50 130 140 150


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 4 → valoare: 60


HIT-uri: 2
MISS-uri: 4
Rată HIT: 33.33%
```

Figura 19:Rezultate obtinute

8.2.2.2 Rezultate obtinute pentru politica Lru

Politica LRU prezintă un comportament similar cu FIFO în faza inițială a testelor de scriere, datorită faptului că memoria cache este inițial goală. WRITE MISS-urile și WRITE HIT-urile apar în aceleași condiții ca în cazul politicii FIFO.

Diferențele dintre cele două politici devin vizibile în scenariile în care cache-ul este complet ocupat, politica LRU reușind să păstreze mai eficient blocurile recent scrise.

```
=== SIMULARE CACHE ===
Politică: LRU

WRITE MISS (LRU) - adresa 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | INVALIDĂ
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

Bloc TAG=0 -> pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 0 → valoare: 10

WRITE MISS (LRU) - adresa 4

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

Bloc TAG=1 -> pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ
```

```
=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 4 → valoare: 20

WRITE MISS (LRU) - adresa 8

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

Bloc TAG=2 -> pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 10 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | INVALIDĂ


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 8 → valoare: 30

WRITE HIT (LRU) - linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | INVALIDĂ


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 0 → valoare: 40

WRITE MISS (LRU) - adresa 12
```

```
=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | INVALIDĂ

Bloc TAG=3 -> pus în linia 3

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 20 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | TAG=3 | Bloc: 50 130 140 150


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 12 → valoare: 50

WRITE HIT (LRU) - linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 40 10 20 30
Linia 1 | TAG=1 | Bloc: 60 50 60 70
Linia 2 | TAG=2 | Bloc: 30 90 100 110
Linia 3 | TAG=3 | Bloc: 50 130 140 150


=== FRAGMENT DIN MEMORIA PRINCIPALĂ ===
Adresa 4 → valoare: 60


HIT-uri: 2
MISS-uri: 4
Rată HIT: 33.33%
```

Figura 20: Rezultate obtinute

8.2.3 Teste de invalidare

Testele de invalidare au fost realizate într-un scenariu în care memoria cache este complet ocupată, pentru a evidenția comportamentul mecanismului de eliminare a blocurilor în condiții reale de funcționare. Înainte de efectuarea operațiilor de invalidare, cache-ul a fost umplut prin accesarea unui număr de adrese distincte egal cu numărul de linii disponibile.

După umplerea cache-ului, au fost efectuate operații suplimentare de acces și invalidare, cu scopul de a observa modul în care liniile cache sunt marcate ca nevalide și modul în care politicile de înlocuire FIFO și LRU gestionează ulterior încărcarea de noi blocuri.

```
R 0
R 4
R 8
R 0
E 4
R 4
```

Figura 21:Fisierul de operatii utilizat

8.2.3.1 Rezultate asteptate

Pentru testele de invalidare, se așteaptă ca operația de invalidare să marcheze corect linia corespunzătoare din memoria cache ca fiind nevalidă. După invalidarea unui bloc, accesarea ulterioară a aceleiași adrese ar trebui să conducă la apariția unui MISS, deoarece blocul respectiv nu mai este prezent în cache și trebuie reîncărcat din memoria principală.

Având în vedere faptul că invalidarea este realizată explicit asupra unui bloc specificat, nu se așteaptă diferențe între politicile de înlocuire FIFO și LRU în cadrul acestui tip de test. Politica de înlocuire nu este implicată în selectarea blocului eliminat, deoarece decizia este impusă direct prin operația de invalidare.

Diferențele dintre FIFO și LRU sunt așteptate să apară doar în scenariile de înlocuire automată, atunci când memoria cache este complet ocupată și este necesară selectarea unei linii victimă ca urmare a unui MISS.

8.2.3.2 Rezultate obtinute

Rezultatele obținute în urma testelor de invalidare confirmă comportamentul așteptat al memoriei cache. Liniile invalidate sunt marcate corect ca nevalide, iar accesarea ulterioară a adreselor corespunzătoare generează MISS-uri, blocurile fiind reîncărcate din memoria principală.

Statisticile obținute pentru politicile FIFO și LRU sunt identice în cadrul testelor de invalidare. Acest comportament este justificat de faptul că operația de invalidare elimină explicit un bloc specificat, fără a implica mecanismul automat de selecție a liniei victimă. În consecință, politica de înlocuire nu influențează rezultatul acestor teste.

Rezultatele obținute demonstrează funcționarea corectă a mecanismului de invalidare și confirmă faptul că diferențele dintre politicile FIFO și LRU trebuie analizate în cadrul testelor de înlocuire automată, realizate pe un cache complet ocupat.

```
=== SIMULARE CACHE ===
Politică: FIFO

MISS (FIFO) - pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ

HIT (FIFO) - linia 0
EVICT (FIFO) linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ

MISS (FIFO) - pus în linia 1
```

```
=== SIMULARE CACHE ===
Politică: LRU

MISS (LRU) - pus în linia 0

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (LRU) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | INVALIDĂ
Linia 3 | INVALIDĂ

MISS (LRU) - pus în linia 2

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ

HIT (LRU) - linia 0
EVICT (LRU) linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | INVALIDĂ
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ
```

```
MISS (FIFO) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ


HIT-uri: 1
MISS-uri: 4
Rată HIT: 20.00%
```

```
MISS (LRU) - pus în linia 1

=== STAREA CURENTĂ A CACHE-ULUI ===
Linia 0 | TAG=0 | Bloc: 0 10 20 30
Linia 1 | TAG=1 | Bloc: 40 50 60 70
Linia 2 | TAG=2 | Bloc: 80 90 100 110
Linia 3 | INVALIDĂ


HIT-uri: 1
MISS-uri: 4
Rată HIT: 20.00%
```

Figura 22:Fisierele de rezultate obtinute pentru cele doua politici de inlocuire

9.Concluzie

Testele efectuate evidențiază faptul că politica LRU oferă, în general, o rată de hit mai bună decât FIFO, datorită păstrării blocurilor accesate recent. Politica FIFO este mai simplă, dar poate elimina date încă utilizate. În cazul testelor de scriere și eviction realizate manual, cu memoria inițial goală, ambele politici înregistrează aceleași rate de hit și miss, deoarece nu există blocuri anterioare care să influențeze decizia de înlocuire.

10.Bibliografie

https://www.geeksforgeeks.org/computer-organization-architecture/cache-memory-in-computer-organization/

https://www.cs.ubbcluj.ro/~rares/course/os/res/book/AGestiuneaMemoriei.pdf

https://users.utcluj.ro/~baruch/book_ac/AC-Memoria-Cache.pdf

https://www.geeksforgeeks.org/computer-organization-architecture/write-through-and-write-back-in-cache/?utm_source=chatgpt.com